

# rebuild-kit

A generator skill that converts a legacy application into an agent-executable rewrite workspace — specs, backlog, and verification as one artifact, consumed by Claude Code and taught to the humans running it.

| DOCUMENT          | VERSION / STATUS | DATE       | SKILL NAME            | REWRITE EXECUTOR |
|-------------------|------------------|------------|-----------------------|------------------|
| Architecture Spec | 0.6 · Draft      | 2026-08-08 | rebuild-kit (working) | Claude Code      |

REVISION LOG

**v0.6** — orchestration substrate:

- U9** Dynamic workflow orchestration — generation, audit, and milestone execution run as Claude Code dynamic workflows: parallel fresh-context subagents, enumerated coverage, worktree-isolated execution, rerunnable orchestration artifacts (§6)

**v0.5** — human learning layer:

- U8** Field guide — a generated, human-consumable projection of the same evidence base: system walkthroughs, then-vs-now, ruling briefs and gate packets wired into the loop (§13)

**v0.4** — rewrite-root structure:

- U7** Rewrite-root layout — legacy and modern as sibling trees under one session root; pinned legacy ref, twin-boot differential verification, CLAUDE.md structure contract (§7)

**v0.3** — problem-context intake:

- U6** Problem brief intake — human-supplied defects, pain points, and goals captured before generation; propagates to fidelity (REPAIR), replay divergence, prioritization, and audit (§4)

**v0.2** — five upgrades over the base concept:

- U1** Differential replay verification — real captured traffic becomes the acceptance bar (§9)
- U2** Runtime evidence ingestion — logs, APM, and analytics drive priorities and fixtures (§5, P2)
- U3** Data migration as a first-class workstream — census, transforms, reconciliation (§10)
- U4** Walking skeleton + risk-gated milestones — thin slice first, human sign-off on risk (§11)
- U5** Adversarial spec audit — fresh-context falsification before the workspace ships (§12)

## 1 Concept & North Star

rebuild-kit runs one directory above the legacy application. From there it ingests the legacy tree — source, schema, runtime evidence, and a human-supplied problem brief — and scaffolds a **rewrite root**: the legacy app pinned in place as read-only evidence, a sibling tree where the modern app will grow, and around them the specifications, ordered backlog, verification harness, and project memory that drive the rewrite. A Claude Code session opened at the root becomes the rewrite agent. Because the consumer is an agent, the docs and the prompts are the same thing.

**Primary and secondary goals.** The primary goal is unchanged and non-negotiable: create the structure and specifications for the rewrite, consumed by Claude Code. The secondary goal is knowledge transfer: teach the humans running the skill how the legacy app works and how the new app will work. The second is served by a generated field

guide (§13) that *projects* the same evidence base into human-readable form — it never forks it, so the agent artifacts stay exactly as agent-shaped as before.

**Orchestration substrate.** The heavy phases and milestone execution run as Claude Code *dynamic workflows* (§6): parallel, fresh-context subagents with the coordination itself captured as a rerunnable artifact. Analysis coverage becomes enumerated structure rather than a single agent's stamina, and the audit's independence becomes a property of the architecture rather than a procedure.

**Scope.** rebuild-kit targets applications small enough to rewrite whole — where both trees fit comfortably under one root and a full rewrite with a clean cutover is realistic. Strangler-scale systems, incremental facades, and long-lived parallel operation are explicitly out of scope; the sibling-tree structure is the load-bearing assumption throughout.

**Scope boundary.** The skill generates; it never rewrites. Its output ends at the scaffolded root — specs, backlog, harness, guide, and an empty modern/ tree with its CLAUDE.md — and the only time it runs again is spec-patch mode, regenerating structure after a human ruling. The rewrite itself is performed later, by ordinary Claude Code sessions opened at the root with no skill installed, and it can never proceed past a gate without human sign-off.

**Definition of success.** The new implementation passes the full verification stack (§9) on all in-scope behavior, and every deviation from legacy behavior is sanctioned — a REPAIR or FREE disposition tracing to the problem brief, or an explicit human ruling. The rewrite is complete when the ledger shows every work order green and every gate signed.

**Degraded mode.** Three subsystems are evidence-dependent (runtime ingestion, data census, trace capture). Absent that evidence the pipeline still runs code-only, but affected claims are capped at lower confidence, verification falls back to derived tests, and the workspace records which upgrades are inactive — so the executor and its humans know exactly what they are trusting.

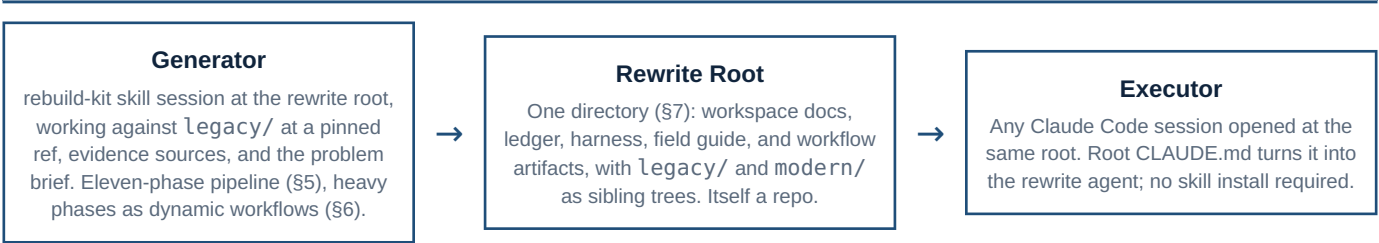
## 2 Design Principles

---

1. **Evidence or it doesn't ship.** Every behavioral claim cites file:line, a captured trace, or a problem-brief entry. Claims that cannot be evidenced are demoted to ASK, never silently asserted.
2. **Fidelity is explicit.** Every spec item carries FIXED, REPAIR, FREE, or ASK — preventing slavish porting of accidental complexity, creative drift on load-bearing logic, and faithful reproduction of known defects.
3. **Executable acceptance.** Replay sets and generated tests define done. Prose never does.
4. **Machine formats at the boundaries.** OpenAPI, JSON Schema, DDL, and payload fixtures instead of prose descriptions — the executor validates against them rather than interpreting them.
5. **Negative space is documented.** Dead code, deprecated paths, and bug workarounds go in a do-not-port list with evidence, or the agent will faithfully rebuild the cruft.
6. **Context economy.** Every artifact is sized for an agent context window; work orders are self-contained and declare their own reading list.
7. **Independent verification.** Specs are attacked by fresh-context auditors, and graded by traces the generator did not author — breaking the self-grading loop.
8. **Effort follows usage — and pain.** Runtime evidence and the problem brief order the backlog; the endpoints carrying the traffic and the defects motivating the rewrite ship first.
9. **Sanctioned change only.** Every deviation from legacy behavior traces to a problem-brief entry or a human ruling. Everything else is drift, however well-intentioned.
10. **One root, two trees.** legacy/ is read-only evidence at a pinned ref; modern/ is the sole write target; everything cites root-relative paths. The structure is the safety rail.

- 11. **One evidence base, two renderings.** The agent artifacts are the source of truth; the field guide is generated from them — same citations, same uncertainty, never hand-forked. It teaches only what survived the audit, and shows open questions as open.
- 12. **Orchestration is an artifact.** Coverage is enumerated, never assumed; verification is woven into the fan-out; and the coordination itself is written down, kept, and rerunnable — reviewed like code, because it is.

### 3 System Overview



**Human lane (throughout):** place the legacy checkout · supply the problem brief · grant evidence access and approve PII scrubbing · choose the target stack · resolve ASK items · sign risk gates and the expected-divergence manifest · own migration rehearsal and cutover. Humans decide; agents execute and verify — and the field guide (§13) keeps the deciders educated.

The generator and executor never share a session. Their only interface is the root, whose schema (references/ schema.md in the skill) is the contract between them. The root is itself a git repository, so the entire rewrite — specs, ledger, decisions, and the modern tree — is independently reviewable: it takes pull requests from humans and is regenerated section-by-section as questions get resolved.

### 4 Intake — The Problem Brief U6

A rewrite exists because something is wrong. Without that context the generator treats every observed behavior as ground truth: a defect with a source citation and a matching trace earns a FIXED tag, and the rewrite faithfully reproduces the problems it was commissioned to solve. So before any analysis runs, the generator conducts a structured intake interview with the humans who commissioned the rewrite and writes docs/problem-brief.md.

#### What the interview captures

- **Motivation** — why rewrite, why now.
- **Known defects** — bugs users live with, each with a reproduction or affected area.
- **Pain points** — performance, reliability, operability, and UX complaints.
- **Architectural grievances** — where the current design fights change.
- **Target goals & constraints** — what the new system must achieve: scale, SLOs, team shape, operability.
- **Non-goals** — improvements explicitly out of scope for this rewrite.

Every entry gets an ID (PB-*nnn*) plus provenance — who reported it, severity, affected area. Human testimony thereby becomes the third evidence class alongside source citations and traces, citable from any artifact in the workspace.

#### How the brief propagates

| DOWNSTREAM CONSUMER | EFFECT OF THE BRIEF |
|---------------------|---------------------|
|---------------------|---------------------|

|                                  |                                                                                                                                                                                                                                                  |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Fidelity taxonomy</b> (§8)    | Behaviors matched to a defect entry are dispositioned REPAIR, not FIXED: the legacy behavior is documented for recognition, the target behavior is specified — by the brief, or by an ASK ruling if the brief names the problem but not the fix. |
| <b>Verification</b> (§9)         | Each REPAIR generates an entry in the expected-divergence manifest, so replay passes intentional fixes instead of flagging them as regressions.                                                                                                  |
| <b>Backlog &amp; gates</b> (§11) | Pain-weight joins usage-weight in prioritization; brief severity feeds risk scores; areas the brief flags as fragile raise gate thresholds.                                                                                                      |
| <b>Target architecture</b>       | Grievances, goals, and constraints are the requirements input to the human's target-stack decision recorded in <code>modern/CLAUDE.md</code> , and steer every FREE choice the executor makes.                                                   |
| <b>Negative space</b>            | Entries marked "must not survive the rewrite" land in <code>do-not-port.md</code> directly, with human provenance.                                                                                                                               |
| <b>Audit</b> (§12)               | Problem-coverage check: every PB entry must hold a disposition — a REPAIR behavior in some work order, a do-not-port entry, an NFR target, or an explicit out-of-scope ruling. Undispositioned entries block workspace assembly.                 |

#### SCOPE DISCIPLINE — THE BRIEF IS ALSO THE WHITELIST

A problem brief invites a failure mode: an agent that over-rotates into fixing everything it dislikes. So the brief cuts both ways. FREE grants mechanism freedom, never feature freedom; any deviation from legacy behavior must trace to a PB ID or a human ruling. When the executor believes some unsanctioned legacy behavior is wrong, it files a PB *proposal* to open-`questions.md` — it does not build the fix.

## 5 Generator Pipeline

Eleven phases, run in order, each producing named artifacts. Deterministic work is pushed into bundled scripts so tokens are spent on judgment, not enumeration; the fan-out-heavy phases run as dynamic workflows (§6). Phases marked **Un** implement the v0.2–v0.6 upgrades.

| PHASE                                       | PURPOSE                                                                                                                                                                                                                                             | KEY OUTPUTS                                                                                                                                                                                                                                     |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>P0</b> Intake & scaffold<br><b>U6 U7</b> | Detect or scaffold the rewrite root: locate the legacy subdirectory, pin <code>legacy_ref</code> , mark the tree read-only, create the layout. Then the structured interview capturing defects, pain points, grievances, goals, and non-goals (§4). | <code>rebuild.json</code> (layout block), root <code>CLAUDE.md</code> skeleton, <code>docs/problem-brief.md</code> (PB- <code>nnn</code> register); seed entries for <code>do-not-port</code> and NFR targets.                                  |
| <b>P1</b> Static inventory                  | Deterministic extraction against <code>legacy/</code> at the pinned ref: module & dependency graph, route/job inventory, DB schema dump, complexity and churn hotspots.                                                                             | <code>inventory.json</code> , <code>hotspots.md</code> — via AST tooling ( <code>ts-morph</code> for TS; generic parsers otherwise), not token-burning file reads. The inventory is also the enumeration the extraction workflow fans out over. |
| <b>P2</b> Runtime evidence <b>U2</b>        | Ingest access logs, APM, analytics, error trackers. PII-scrub payloads at intake.                                                                                                                                                                   | <code>usage-weights.json</code> , <code>perf-envelopes.json</code> , scrubbed payload corpus, zero-traffic report ( <code>do-not-port</code> evidence).                                                                                         |
| <b>P3</b> Architecture & domain recon       | Map subsystems and boundaries; extract domain entities, invariants, and vocabulary.                                                                                                                                                                 | <code>docs/00-overview.md</code> , <code>docs/domain/</code> .                                                                                                                                                                                  |

|                                             |                                                                                                                                                                                                    |                                                                                                               |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>P4</b> Behavioral extraction             | Per-feature rules, edge cases, and error paths mined from source; every claim cited and confidence-tagged; behaviors matching brief entries dispositioned REPAIR.                                  | Draft feature specs — the raw material for work orders.                                                       |
| <b>P5</b> Contract extraction               | Freeze the boundaries in machine-checkable form.                                                                                                                                                   | contracts/: OpenAPI, event & JSON Schemas, DDL, integration notes, golden fixtures.                           |
| <b>P6</b> Data census & migration <b>U3</b> | Probe prod-shaped data for reality; plan the move.                                                                                                                                                 | docs/migration/: census report, schema mapping, transform + reconciliation plan (§10).                        |
| <b>P7</b> Replay harness <b>U1 U7</b>       | Build the twin-boot harness; capture and normalize traces; record legacy golden outputs at the pinned ref; define diff rules and expected divergences; assign replay sets per work order.          | verification/replay/: traces, diff-rules.yaml, expected-divergences.yaml; verification/harness/ runners (§9). |
| <b>P8</b> Backlog synthesis <b>U4</b>       | Cut specs into work orders; score risk; weight by usage and pain; order topologically; define milestones and gates. Milestone 0 is always the walking skeleton.                                    | backlog.md, ledger.json, docs/features/W0-*.md.                                                               |
| <b>P9</b> Adversarial audit <b>U5 U9</b>    | Adversarial workflow: falsification pass over every FIXED claim; branch-coverage hunt; problem-coverage check; demotions applied.                                                                  | audit/ report + metrics (§12).                                                                                |
| <b>P10</b> Field guide <b>U8</b>            | Render the human layer from the <i>audited</i> artifacts: walkthroughs, then-vs-now, flow storyboards from annotated traces, glossary; wire ruling-brief and gate-packet generation into the loop. | guide/ tree (§13). Workspace assembly follows.                                                                |

## 6 Orchestration — Dynamic Workflows **U9**

rebuild-kit's heavy lifting runs on Claude Code **dynamic workflows**: Claude writes the orchestration for a task on the fly, and a runtime executes it — fanning the work across many parallel subagents, each with a clean context window and one focused job, then converging and cross-checking their results. What the playbooks used to state as instruction ("analyze every module; skip nothing") becomes enumerated structure: the workflow walks the full inventory, so coverage is a property of the plan rather than a hope about a single agent's attention span. This section specifies *what* runs as a workflow and what each pattern guarantees; the orchestration itself is written at run time and is deliberately not specified here.

Three inherited properties do the work. **Fresh contexts per subagent** eliminate coverage decay — the failure mode where a long-running analysis quietly tires after the thirty-fifth file of fifty. **Adversarial cross-checking is the native pattern** — independent agents attack findings before they are accepted, which is exactly the posture this design demands of its audit. And **the orchestration is itself an artifact** — a script that can be read, kept, and rerun, which turns every analysis run into something reviewable and reproducible rather than a one-time performance.

| WORKFLOW                         | SHAPE                                                                                                                                                                                         | WHAT THE PATTERN BUYS                                                                                                         |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>Extraction</b> (P1, P3–P5)    | Fan out over the P1 inventory — one subagent per module, route group, or schema object; paired extract-and-verify agents so a claim is independently confirmed before it enters a draft spec. | Enumerated, auditable coverage of the legacy tree; claim-level verification at the source instead of only at the end.         |
| <b>Adversarial audit</b> (P9)    | Independent adversaries per claim cluster plus branch-coverage hunters, every subagent starting from a clean context with only the legacy tree, brief, and draft specs.                       | The audit's isolation requirement becomes structural rather than procedural, and audit depth stops depending on app size.     |
| <b>Milestone execution</b> (§11) | The ledger's dependency graph schedules the full frontier of unblocked work orders across worktree-isolated subagents; merges land only through a green harness run.                          | Throughput between gates without touching the control points; isolation keeps parallel work orders from trampling each other. |
| <b>Spec patch &amp; re-audit</b> | A human ruling triggers a targeted slice: re-extract only the affected specs, re-audit what was touched, unblock the ledger.                                                                  | Cheap, surgical regeneration — rulings propagate in minutes, not via a full pipeline rerun.                                   |
| <b>Guide render</b> (P10)        | Per-chapter parallel rendering from the audited artifacts; briefs and packets generated on demand during execution.                                                                           | The field guide stays current at negligible cost, so it actually gets regenerated.                                            |

## Workflow artifacts

The orchestration scripts Claude writes for this rewrite are retained under `workflows/` in the root — versioned, human-reviewable, and rerunnable. A regeneration months later re-executes the same orchestration against the same pinned ref; a reviewer can read exactly how the analysis was divided, verified, and merged. Workflows also choose the intelligence per role — mechanical passes on faster models, judgment and adversarial roles on stronger ones — and those choices land in the run record alongside the results. Interrupted workflows resume, which composes cleanly with the root's own resume semantics: `rebuild.json` says where the rewrite stands, the workflow runtime says where the current run stands.

## Boundaries and proportionality

Workflows accelerate the spans *between* human control points; they never cross them. Gates, open ASK items, and the expected-divergence manifest pause or bound every workflow, and `ledger.json` remains the single source of truth that workflows read and write — the orchestration layer holds no state of its own that matters. Proportionality is a design rule, not an afterthought: dynamic workflows consume substantially more tokens than a plain session, so the skill scales orchestration to the target — every workflow-shaped phase has a serial fallback, and small targets may run the entire pipeline that way. The workflow versions exist for when *scale*, not judgment, is the constraint. (Availability: current Claude Code, v2.1.154 or later, on paid plans.)

## 7 The Rewrite Root U7

The structural convention: the skill runs one directory above the legacy application, and everything the rewrite needs lives under that one root — legacy and modern as sibling trees, with the workspace around them. One session root, two application trees, every path root-relative.

```

rewrite-root/      # session root – generator and executor both run HERE
├─ CLAUDE.md       # root contract: layout map, roles, read/write rules, executor loop,
│                  # gate + escalation rules
├─ rebuild.json    # layout block: legacy/modern dir names, legacy_ref SHA, target stack,
│                  # skill version – also the marker that identifies a rewrite root
├─ backlog.md      # human-readable plan: milestones M0..Mn, ordered work orders
├─ ledger.json     # machine state: W0 status, verification results, gate approvals
├─ docs/
│   ├── 00-overview.md    # system map, subsystem boundaries
│   ├── problem-brief.md  # PB-nnn register: defects, pains, goals, non-goals (§4)
│   ├── domain/           # entities, invariants, glossary
│   ├── features/W0-014-*.md # self-contained work orders (§8)
│   ├── contracts/        # openapi.yaml, schemas/, ddl.sql, fixtures/
│   ├── migration/        # mapping.md, census.md, reconciliation.sql (§10)
│   ├── do-not-port.md    # negative space, each entry with evidence
│   └─ open-questions.md  # ASK register + PB proposals; entries link the W0s they block
├─ verification/
│   ├── replay/           # traces/*.jsonl, diff-rules.yaml, expected-divergences.yaml
│   ├── characterization/ # generated tests + golden fixtures
│   └─ harness/           # run-legacy.sh, run-modern.sh, diff-run.sh – twin-boot (§9)
├─ audit/           # discrepancy report, spec + problem coverage metrics (§12)
├─ workflows/       # orchestration scripts written for this rewrite – rerunnable,
│                  # reviewed like code (§6)
├─ guide/           # field guide – generated human layer, never hand-edited (§13)
├─ legacy/          # the existing app – READ-ONLY, pinned at legacy_ref
└─ modern/          # the new app – sole write target; own CLAUDE.md with target-stack
                   # conventions and architecture rules

```

## The layout block

`rebuild.json` makes the structure declarative rather than assumed: it records the actual directory names (defaults `legacy/` and `modern/`; real repo names work fine), the pinned `legacy_ref`, the chosen target stack, and the skill version that generated the root. Every script and every doc resolves paths through it. Its presence is also the resume signal: a skill session that finds `rebuild.json` knows it is inside an existing rewrite and routes to resume or spec-patch mode instead of scaffolding from scratch.

## Rules of engagement

- **Legacy/ is read-only evidence.** Enforced three ways: by contract in the root `CLAUDE.md`; mechanically (the scaffold strips write permissions, and a pre-commit hook rejects any diff under `legacy/`); and by pinning — `legacy/` is a git submodule or SHA-recorded clone fixed at `legacy_ref`.
- **The pin makes citations durable.** Every `file:line` citation in the workspace is valid against `legacy_ref` forever, and legacy golden outputs recorded by the harness stay valid indefinitely. If upstream legacy keeps moving, a small script diffs upstream against the pin and reports which cited ranges changed — staleness is detectable, cheaply, without any monitoring machinery.
- **modern/ is the sole write target.** The executor writes application code only there, and workspace state only to the ledger, open-questions, and its own notes. Everything else it treats as spec.
- **Root-relative paths everywhere.** Citations read `legacy/src/auth/reset.ts:88-114`; implementations land in `modern/src/...`. No ambiguity about which tree is being read versus written, and every artifact is portable with the root.

## The CLAUDE.md hierarchy

Claude Code reads `CLAUDE.md` files natively, which is precisely how the executor "knows" the structure with zero skill install. The **root CLAUDE.md** carries the structure map and rules above, the executor loop (§11), and gate and escalation



protocol. **modern/CLAUDE.md** carries the target-stack decision, coding conventions, and architecture rules for the new tree — generated from the problem brief's goals and the human's stack choice, so that FREE decisions land consistently. The legacy tree gets no CLAUDE.md: it is evidence, not a place to work, and the skill never writes into it.

Git strategy

The root is one repository: specs, ledger, decisions, and modern/ in a single reviewable history, with legacy/ as a pinned submodule or an ignored clone whose SHA lives in rebuild.json. Spec changes and code changes arrive as commits and can take pull requests from humans. At cutover, modern/ graduates — subtree-split into its own repository — and the root remains behind as the complete, audited decision record of the rewrite.

8 Work Order Schema & Fidelity Taxonomy

Work orders are the unit of execution: self-contained, dependency-declared, and sized to a stated context budget so the executor loads only the order plus its linked contracts.

```
# docs/features/W0-014-password-reset.md (excerpt)
id: W0-014          depends_on: [W0-003, W0-007]      milestone: M2
risk: 0.62 (inferred-claim ratio 0.3, complexity high, legacy coverage none)
usage_weight: 0.041  pain_weight: 0.18  context_budget: ~350 lines  gate: false

behaviors:
- statement: Reset tokens expire after 30 minutes; expired tokens return
  the SAME error body as invalid tokens (deliberate non-disclosure).
  fidelity: FIXED
  evidence: [legacy/src/auth/reset.ts:88-114, trace: replay/traces/auth-041.jsonl]
- statement: Reset email is sent synchronously in-request; times out under
  provider latency spikes (PB-012, severity high).
  fidelity: REPAIR – target: enqueue via transactional outbox (ruled OQ-003)
  evidence: [legacy/src/auth/reset.ts:131-140, trace: auth-017]  divergence: ED-012
- statement: Token storage mechanism (currently MD5 in a legacy table).
  fidelity: FREE – outcome required (single-use, expiring); mechanism open.
- statement: Rate limit of 3 requests/hour appears in code but a bypass
  header is honored in legacy/src/middleware/rl.ts:41. Intent unclear.
  fidelity: ASK – open-questions.md#OQ-007 (blocks: none; flags gate review)

acceptance:
  replay_set: auth-reset-*.jsonl (14 traces; ED-012 applies)
  tests: characterization/auth/reset.spec.ts
  escalation: consult legacy/src/auth/reset.ts:60-140 only if spec ambiguity found
```

| TAG    | MEANING                                                               | EVIDENCE BAR                                                                                                        | EXECUTOR BEHAVIOR                                                                                   |
|--------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| FIXED  | Behavior must match legacy observably.                                | Source citation <b>and</b> trace or characterization coverage.                                                      | Implement exactly; any deviation fails verification.                                                |
| REPAIR | Legacy behavior is real and evidenced, but designated for correction. | Evidence of the current behavior <b>plus</b> a PB citation; target behavior ratified by the brief or an ASK ruling. | Implement the target behavior; verified against the expected-divergence entry, not trace match.     |
| FREE   | Outcome required; mechanism open.                                     | Rationale note for why fidelity is not required.                                                                    | Choose the idiomatic target-stack approach per modern/CLAUDE.md; record the choice in ledger notes. |



|     |                                           |                                                       |                                                                              |
|-----|-------------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------|
| ASK | Ambiguous, conflicting, or inferred-only. | The conflict itself, with citations for each reading. | Never guess. Log to open-questions.md, block dependents, continue elsewhere. |
|-----|-------------------------------------------|-------------------------------------------------------|------------------------------------------------------------------------------|

## 9 Verification Stack U1 U7

Acceptance is layered; each layer catches what the one below cannot.

| LAYER                                   | MECHANISM                                                                                                                                                                                                                                                                                                         | CATCHES                                                                                                                                   |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>L1</b> Contract validation           | Responses validated against OpenAPI/JSON Schema; DDL diffed against target schema.                                                                                                                                                                                                                                | Shape drift, missing fields, type regressions.                                                                                            |
| <b>L2</b> Characterization tests        | Generated tests + golden fixtures derived during analysis. Fast inner loop.                                                                                                                                                                                                                                       | Logic errors against the spec as written.                                                                                                 |
| <b>L3</b> Twin-boot differential replay | Both trees run from the same root: the harness boots legacy/ and modern/ on identical seeded fixtures and drives identical inputs, diffing responses <i>and</i> post-run state (DB dumps, emitted events, outbound calls) under <code>diff-rules.yaml</code> normalization — timestamps, generated IDs, ordering. | <b>Spec misreads.</b> The layer the generator cannot game, because the expected values come from executing legacy, not from the analysis. |

### WHY L3 EXISTS — BREAKING THE CIRCULARITY

Without replay, the same analysis that writes the specs also generates the tests that grade the rewrite: a confident misreading passes cleanly, tests green, behavior wrong. Executing the legacy tree itself reintroduces ground truth from outside the analysis loop — and the sibling layout makes that a local operation instead of an infrastructure project. It is the single largest contributor to rewrite success probability in this design.

**Expected divergences.** Replay's premise is that old and new agree — except where disagreement is the point. `expected-divergences.yaml` maps trace patterns to PB IDs and the specified replacement behavior; the differ passes those entries only when they diverge *as specified*, and any divergence not in the manifest remains a failure. The manifest is human-signed, and doubles as the authoritative changelog of every intentional behavior change in the rewrite.

**Input tiers.** **T1** — captured production traffic, when available (highest realism; optional, never assumed). **T2** — the workhorse: scripted sessions and generated input sets driven through the twin-boot harness; legacy golden outputs are recorded once per input set and cached — the pin makes the cache valid indefinitely — so the inner loop boots only modern/. **T3** — statically derived request/response pairs, marked provisional; T3-only evidence counts toward L2, never L3, and is flagged in the ledger. **Per-WO acceptance bar:** 100% of the assigned replay set within diff rules — expected divergences diverging as specified — plus all characterization tests green.

## 10 Data Migration Workstream U3

More rewrites die in the data than in the code, so migration is generated as ordinary, verifiable work orders — not an appendix.

- **Dirty-data census.** Generated queries probe prod-shaped data for the ways reality violates the schema: nulls in "required" columns, orphaned foreign keys, encoding anomalies, out-of-range enums, duplicate values under "unique" intent, timezone-naive datetimes. Output: `census.md` with counts and scrubbed samples.

- **Mapping & transforms.** `mapping.md` defines old → new schema mapping and an explicit policy per dirty class — repair, quarantine, or drop-with-log. Policies are ASK items until a human ratifies them.
- **Reconciliation as acceptance.** Migration WOs pass only when reconciliation checks do: row counts, per-column checksums, and stratified sampled field-level diffs between source and target. The twin-boot harness runs both schemas locally, so census, transform runs, and reconciliation are inner-loop operations, not environment requests.
- **Rehearsal and cutover.** A full dry run against a production snapshot is a gated milestone. Cutover sequence and rollback plan are documented in the workspace and owned by humans.

## 11 Execution Model U4

---

### The executor loop

1. Read the root `CLAUDE.md` and `ledger.json`; select the highest-priority work order whose dependencies are satisfied and which is not blocked by a gate or an open ASK — or, under a workflow, the full frontier of such orders.
2. Load only that work order plus its linked contracts (the WO declares its context budget).
3. Implement in `modern/` per fidelity tags — exact on `FIXED`, target behavior on `REPAIR`, idiomatic on `FREE`. Enter `legacy/` only at the cited ranges in escalation pointers, never for bulk reading.
4. Run L1 and L2 locally; run the WO's replay set through the twin-boot harness (L3), expected divergences included.
5. Green: mark done in the ledger, record `FREE` choices made, commit.
6. Red: fix the implementation. If the *spec* appears wrong — or the executor believes an unsanctioned legacy behavior should change — never act silently: file it to `open-questions.md` (as a discrepancy or a PB proposal), generating its ruling brief (§13), and set the WO to `awaiting_ruling` if blocked.
7. At gate WOs: stop, emit the gate packet (§13), and request human sign-off; the ledger records approver and timestamp.
8. Close each milestone with a full-suite regression replay — and human review of any new expected-divergence entries — then refresh the guide's as-built pages from the ledger before starting the next.

### Parallel execution under workflows

Between control points, milestone execution runs as a dynamic workflow (§6): the ledger's dependency graph schedules every unblocked work order across `worktree`-isolated subagents, and each subagent's result merges into `modern/` only through a green harness run. Conflicts, cross-WO discoveries, and shared-helper questions file to `open-questions.md` like any other discrepancy — parallelism changes throughput, never semantics. Gates and open ASK items remain hard boundaries: the workflow pauses there, and serial execution remains fully valid for small targets.

### Milestone 0 — the walking skeleton

Before bulk work, the executor builds one thin end-to-end slice of the highest-usage flow — entry, auth, one core action, persistence, response — running in `modern/` and passing the harness. This validates the target-stack choices, proves the twin-boot plumbing, and surfaces systemic spec misreads while they cost one work order instead of forty.

### Risk gates

Each WO's risk score is a function of inferred-claim ratio, ASK density, problem-brief severity, cyclomatic complexity, legacy test coverage, churn history, and audit findings. Above threshold, `gate: true` — the executor must halt for human review before the WO can close. This turns ASK from a passive tag into a workflow: open questions block their dependents, human rulings patch the spec, the audit re-runs on touched specs, and the ledger unblocks — all without the executor ever improvising on load-bearing behavior.

## 12 Adversarial Spec Audit U5 U9

The executor will trust the docs completely, so the docs must earn it. Before workspace assembly, the audit runs as an adversarial dynamic workflow (§6): independent fresh-context subagents — given only the legacy tree, the problem brief, and the draft specs, none of the generator's reasoning — with isolation guaranteed by construction rather than by procedure. Four procedures per module:

1. **Falsification.** For every FIXED claim, attempt to construct a counterexample from source: an input, flag state, or code path under which the claimed behavior does not hold.
2. **Coverage hunt.** Enumerate branches, feature flags, and error paths present in source but represented in no spec; each becomes a new spec item or a do-not-port candidate.
3. **Fixture spot-check.** Validate sampled golden fixtures against both the contracts and the source logic.
4. **Problem coverage.** Verify every PB entry holds a disposition — a REPAIR behavior in some work order, a do-not-port entry, an NFR target, or an explicit out-of-scope ruling. Undispositioned entries block assembly.

Verdicts are confirmed | contradicted | unverifiable. Contradicted and unverifiable claims are demoted to ASK automatically. The workspace ships with the audit report and its metrics — percent of claims confirmed, spec branch coverage, problem coverage, demotion rate — so the humans signing gates know how hard the docs were attacked, and how they held up.

## 13 The Field Guide U8

The secondary goal, structurally separated so it can never dilute the primary one. The agent artifacts stay agent-shaped; the field guide is a *generated projection* of the same evidence base into human-readable form — narrative, diagrams, and progressive depth that an agent doesn't need and a person does. It runs as P10, after the audit, so it teaches only claims that survived falsification; every guide statement links the same citations and carries the same fidelity and confidence, and open questions appear as exactly that — open.

```
guide/
├─ 00-orientation.md      # human layer – generated projection, never hand-edited
├─ legacy/                # the 10-minute tour: what this system is, the map, where to start
├─ modern/                # how the legacy app works: per-subsystem chapters, narrative + diagrams
├─ flows/                 # how the new app will work; fills in as-built as milestones close
├─ then-vs-now.md         # end-to-end storyboards: sequence diagram + an annotated real trace
├─ decisions.md           # side-by-side: what changes (REPAIR/PB), what holds (FIXED), what's open
├─ glossary.md            # readable changelog: divergence manifest + rulings, with rationale
└─ briefs/                # domain vocabulary from recon
                          # just-in-time: ruling briefs (OQ-*) and gate packets (W0-*)
```

### Teaching both systems

**Legacy chapters** tell the how-it-works story per subsystem — architecture narrative, the main flows, and the archaeology ("why is it like this"), with the weirdness explained through its PB entries and code evidence. **Modern chapters** start as as-designed — target architecture, what each subsystem becomes — and fill in as-built from the ledger as milestones close, including the FREE choices the executor actually made. **Flow storyboards** reuse the replay corpus as teaching material: a sequence diagram beside a real captured trace, annotated step by step — an actual password reset walking through the system beats any prose description. **then-vs-now.md** is the rewrite at a glance: per subsystem, what changes and why (traced to PB IDs), what holds, what's still undecided.

## Just-in-time briefs — education inside the workflow

The guide is not only chapters to read; it meets humans at the moments the process needs them. When the executor files an ASK, it generates a **ruling brief**: the question, the evidence for each reading, the affected flows, and the options with their consequences — one page, everything needed to rule well without opening the legacy tree. When a gate WO halts, it emits a **gate packet**: what the work order does, its risk drivers, what to inspect, and the relevant traces. This is the secondary goal feeding the primary: the six human touchpoints all demand understanding, and briefs deliver it exactly when it's needed — rulings get faster *and* better.

## Lifecycle and honesty rules

The guide is a build artifact: generated in P10, regenerated when rulings patch specs and when milestones close, and never hand-edited — a correction belongs in the spec, and the guide re-renders from it, so a second source of truth cannot emerge. It teaches nothing beyond the audited spec, surfaces confidence honestly, and marks unbuilt sections as designed-not-built. The markdown-plus-Mermaid tree is the canonical form; an optional rendered handbook (HTML or PDF) can be produced from it for distribution outside the repo. Beyond the rewrite itself, this is the knowledge-transfer instrument: rewrites traditionally evaporate institutional knowledge at cutover, leaving a team operating a system nobody deeply knows — here the team that ships modern/ also inherits a guided tour of it, and of everything it replaced.

## 14 Skill Anatomy

```
rebuild-kit/
├── SKILL.md                # orchestrator: mode detection (scaffold | resume | spec-patch),
                           # intake first, then phase + workflow routing, guardrails (<500 lines)
├── references/
│   ├── phases/P0..P10.md  # per-phase playbooks: what to fan out over, what convergence requires
│   ├── templates/         # root CLAUDE.md, modern CLAUDE.md, problem-brief, WO, backlog,
│   │                       # open-questions, audit report, guide chapters, ruling brief, gate packet
│   └── schema.md          # rebuild.json + ledger + WO definitions – the generator/executor contract
├── scripts/
│   ├── scaffold/         # root bootstrap: layout block, legacy pin + read-only, dir scaffold,
│   │                     # pre-commit guard, staleness check vs upstream
│   ├── inventory/        # dep-graph, route map, schema dump (ts-morph for TS; AST fallback)
│   ├── evidence/         # log/APM parsers, PII scrubber, usage weighter
│   ├── replay/           # twin-boot runners, golden-output recorder, trace normalizer,
│   │                     # divergence-aware differ
│   ├── census/           # data census query generators
│   └── guide/            # projection renderer: chapters, mermaid diagrams, briefs;
                           # optional handbook (HTML/PDF)
└── assets/               # workspace scaffold
```

Standard progressive disclosure: metadata always in context, SKILL.md on trigger, phase playbooks read only for the phase at hand, scripts executed without loading. The bundled `scripts/` are deterministic extractors and are distinct from the dynamic-workflow orchestration scripts, which Claude writes at run time and which land in the root's `workflows/` directory (§6) — the playbooks state the *what* for each workflow, never its code. The executor deliberately requires *no* skill — the root CLAUDE.md hierarchy is the executor spec, keeping consumption zero-setup. A thin companion executor skill remains an option later, but the schema contract makes it unnecessary for v1.

## 15 Human Touchpoints & Residual Risk

**Humans stay in the loop at six points:** supplying the problem brief in the intake interview (and ratifying REPAIR target behaviors); placing the legacy checkout, granting evidence access, and approving PII scrubbing; choosing the target

stack; resolving ASK rulings; signing risk gates, milestone reviews, and the expected-divergence manifest; owning migration rehearsal and cutover. Everything else is agent-executed and machine-verified — and every touchpoint arrives with its field-guide brief or packet, so the humans deciding are the humans who understand.

**RESIDUAL RISK — WHAT NO GENERATOR CAN CAPTURE FROM ARTIFACTS**

Undocumented environmental coupling (cron entries, infra quirks, humans-as-integration-glue); behavior that never appeared in any captured or scripted input set; and organizational cutover risk. The nine upgrades target the dominant historical failure modes of rewrites at this scale — self-grading specs, coverage decay in single-context analysis, unweighted effort, data reality, late discovery of systemic misreads, unaudited confidence, faithfully rebuilding the very problems that motivated the rewrite, ambiguity about where evidence ends and the new system begins, and institutional knowledge that evaporates at cutover — raising estimated end-to-end success from roughly 5/10 to ~8.5/10 for the small-to-mid applications this skill targets. The remainder is irreducible from artifacts alone and is exactly where the human lane exists.

16 Build Roadmap

| MILESTONE               | SCOPE                                                                                                                                                                      | EXIT CRITERION                                                                                                                                                                          |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| M1 Static core          | P0 intake + root scaffold (layout block, legacy pin, guards) + problem-brief template; P1, P3–P5 with extraction run as a dynamic workflow; WO schema; workspace assembly. | Intake + code-only run on a real repo yields a scaffolded rewrite root with cited, fidelity-tagged specs — coverage enumerated per file — including REPAIR dispositions from the brief. |
| M2 Execution loop       | P8 backlog + ledger; root + modern CLAUDE.md scaffolds; walking-skeleton milestone pattern; parallel frontier execution.                                                   | An executor session completes Milestone 0 of a small app end-to-end from the root alone.                                                                                                |
| M3 Verification         | Characterization generation; twin-boot harness with golden-output caching; divergence-aware differ.                                                                        | A seeded spec error is caught by twin-boot replay (L3), not by L2 — and a sanctioned REPAIR passes via the divergence manifest.                                                         |
| M4 Evidence & migration | P2 runtime ingestion; P6 census + migration workstream.                                                                                                                    | Backlog reorders under real usage and pain weights; census flags seeded dirty rows; reconciliation WOs pass.                                                                            |
| M5 Hardening            | P9 audit as an adversarial workflow; risk scoring + gates; production trace ingestion (T1) where available.                                                                | Audit demotes a seeded false claim and flags an undispositioned PB entry; a gated WO halts the executor until sign-off.                                                                 |
| M6 Field guide          | P10 projection renderer; legacy/modern chapters + then-vs-now + flow storyboards; ruling briefs and gate packets wired into the loop.                                      | A reviewer resolves a seeded ASK from its ruling brief alone — without opening the legacy tree — and the guide re-renders on the resulting spec patch.                                  |

Each milestone is independently useful: M1 alone already outperforms conventional reverse-engineering docs, and value compounds from there. Build with skill-creator's draft → test → evaluate loop; the exit criteria above double as its test prompts.